SE 413/513: Software Testing

Assignment 3: Integration Testing and System Testing

Total Points: 7.5 **Submission:** GitHub repository and branch

Platform: Python with `pytest`, `flask`, and `requests` is the default and assumed throughout this document. A different programming language/testing framework may be used only with prior instructor approval, and must still demonstrate the equivalent concepts (real components wired together; a system run as a separate process and tested only through its external interface).

1. The Practical Problem

The **Course Registration System** from Assignments 1 and 2 is now a small but complete *system*. `can_register()` and `calculate_registration_fee()` are unchanged; the starter code builds an HTTP API, a service, and a SQLite database around them, plus a `serve.py` that runs the whole system as one process:

```
HTTP client -IP1-> api.py -IP2-> RegistrationService -IP3-> RegistrationRepository -IP4-> SQLite
                                                    `-IP5-> notifier (send_confirmation / send_rejection)
```

Students enroll in courses (`POST /enrollments`), withdraw (`DELETE`), and view their schedule; each course has a limited number of seats, and every decision is sent to the student as a notification (written by `serve.py` to an *outbox* file). `SPEC.md` in the starter is your specification: requirements **REQ-1 to REQ-8**, the HTTP API with its status codes, and the seed data. **You will not modify `src/` or `serve.py`.** Your work is to test how the components work *together* (integration testing) and how the running system behaves *from the outside* (system testing). Put every requirement ID a test checks in that test's docstring.

One known defect. The starter contains exactly one defect, deliberately. It is not visible to unit tests that mock the repository. Section 4(d) asks you to expose and document it — **not** to fix it.

2. GitHub Project and Submission Requirements [0.25 points]

Create a repository, add a branch named exactly `assignment-3`, and give the instructor (GitHub ID `mdkamrulhasan`) access. All work must be committed and pushed to `assignment-3`.

Submit your GitHub branch link, including a reference to your final commit (commit hash or message). Do not update the branch after your final submission — the **latest push timestamp on `assignment-3` is treated as your submission date**; use a different branch if you keep working.

3. Set Up the Starter Project [0.25 points]

```
assignment-3/
|-- src/ (provided: registration, db, repository, service, notifier, errors, api)
|-- serve.py, SPEC.md, requirements.txt, pyproject.toml, README.md (provided)
|-- tests/integration/conftest.py, test_integration.py (yours)
|-- tests/system/conftest.py (provided: the ‘system‘ fixture)
|-- tests/system/test_system.py (yours)
```

Run `pip install -r requirements.txt`, start the system with `python serve.py`, and confirm that `GET /health` answers `{"status": "ok"}` and that one enrollment by hand succeeds (`README.md` has the commands). `pyproject.toml` registers the `integration` and `system` markers and sets `xfail_strict = true`.

4. Integration Testing [3.25 points]

Integration tests assemble **real** components *inside* the test process: the real repository and service on a real SQLite file in `tmp_path`. Only the notifier (IP5, which in production would send email) is replaced, by a test double you write. Mark every test `integration`.

(a) **[0.5]** In `README.md`, list the five integration points IP1–IP5 in a table: the two components, how they communicate (HTTP, Python call + exceptions, SQL), and **one thing the two sides could disagree about** at that point.

(b) **[1] Repository + SQLite (IP4).** In `tests/integration/conftest.py`, write fixtures for a new database (`db.connect()`, `db.create_schema()`, your own small set of students and courses) and a `RegistrationRepository`. Then write at least **four** tests against the real database: (i) `take_seat()` at the capacity **boundary** (the last seat succeeds; one more is refused and changes nothing); (ii) a duplicate enrollment is rejected by the database; (iii) `current_credits()` counts only that student's courses, and is `0` for a student with none; (iv) a transaction that fails part-way leaves neither the seat nor the enrollment behind.

(c) **[0.75] Service + repository + notifier (IP3, IP5).** Write a fake notifier (a small class that records every call in a list) and a `RegistrationService` fixture. Test, checking **both sides** each time — what the database holds *and* what was sent: a successful enrollment (enrolled, seat taken, confirmation with the correct fee); a full course (`CourseFull`, not enrolled, rejection with its reason); and a duplicate enrollment (`AlreadyEnrolled`, no extra seat taken, nothing sent).

(d) **[1] The known defect.** It concerns **withdrawing** from a course (REQ-6). Write an integration test that exposes it, mark it `@pytest.mark.xfail(strict=True, reason="...")` with a reason that names the defect, and leave the code unfixed. In `README.md`, explain in 2–3 sentences why a unit test of `RegistrationService.withdraw()` with a *mocked* repository would pass.

5. System Testing [3.25 points]

System tests run the **whole system as a separate process** — the provided `system` fixture starts `serve.py` with a fresh, seeded database for each test — and observe it **only** through HTTP and the outbox (`system.outbox()`). Do not import anything from `src/` in `tests/system/`. Mark every test `system`.

(a) **[1.25] End-to-end scenarios.** Write two: (i) the main success scenario (a student enrolls, then the response, the schedule, the course's `seats_taken`, and the outbox all agree), and (ii) one alternative flow of your choice (e.g. a missing prerequisite or the credit limit, using the seed data

in `SPEC.md`), again checking every visible consequence.

(b) **[1] Robustness and security (REQ-8).** A parametrized test with at least **five** invalid or hostile `POST /enrollments` bodies (wrong types, missing fields, a non-object body, an oversized or injection-like value, ...): each gets a **JSON** `400`, and nothing changes (no seat taken, nothing in the outbox). Plus a parametrized test that every endpoint except `/health` answers `401` without the API key.

(c) **[1] Concurrency (REQ-2).** SE499 has 3 seats. Ten eligible students (`S01`–`S10`) request it **at the same moment**: one thread per student, released together with a `threading.Barrier`. Assert exactly 3 `201`s and 7 `409`s, `seats_taken == 3`, and 3 confirmations and 7 rejections in the outbox. In `README.md`, explain in 1–2 sentences why this test must be a system test rather than an integration test.

6. Organize and Document the Test Suite [0.5 points]

(a) **[0.25]** Every test carries the `integration` or `system` marker; `README.md` documents the commands for the full suite and for each level (`pytest -m integration`, `pytest -m system`).

(b) **[0.25]** Demonstrate the full suite passes from the project root, with the strict `xfail` of 4(d) as the only non-passing test.

7. Deliverables Checklist *[No additional points]*

Before your final push, confirm the repository has: `src/` and `serve.py` unmodified; your fixtures and tests in `tests/integration/` and `tests/system/`; the provided `tests/system/conftest.py`; `README.md` with the integration-point table (4a), the mocked-repository explanation (4d), the concurrency explanation (5c), and the run commands; and evidence the suite passes (`pytest -v` output).

Recall: submit your GitHub branch link with a reference to your final commit, and do not update the branch afterward.

Guiding principle: Unit tests check each component against what *it* expects; integration tests check that the real components agree with *each other*; system tests check that the running whole does what the *specification* says, from where its users — and its attackers, and its many simultaneous users — stand.